



Nombre: Luis Alberto Vargas Gonzalez

Fecha: 09/02/2026

Actividad y unidad: Unidad 4 A1: Práctica: Funcionamiento del
ataque Dirty Cow.

Clase: Seguridad de Software.

Maestría en Ciberseguridad.

INTRODUCCIÓN.

La seguridad en el núcleo (kernel) de Linux es fundamental para la integridad de millones de sistemas en todo el mundo. Sin embargo, históricamente han surgido fallos que permiten a usuarios sin privilegios tomar el control total del sistema. En esta práctica, analizaremos una de las vulnerabilidades más famosas y críticas de la última década: **Dirty COW (CVE-2016-5195)**, explorando su identificación y documentación técnica mediante herramientas especializadas en Kali Linux como searchsploit.

Cabe señalar que esta vulnerabilidad a nivel de kernel de Linux no es la única que ha surgido en los últimos 3 años, existiendo junto con otras, como, por ejemplo: Dirty Pipe que es la sucesora de Dirty Cow , y de la familia Dirty , User-After-Free (UAF) recientemente descubierta, y además vulnerabilidades en librerías que están ligadas a las importantes librerías de Linux como xz utils la cual permitía ingresar a un atacante remoto autenticarse en servidores SSH sin los permisos necesarios.

DEFINICIÓN Y EXPLICACIÓN DE ATAQUE.

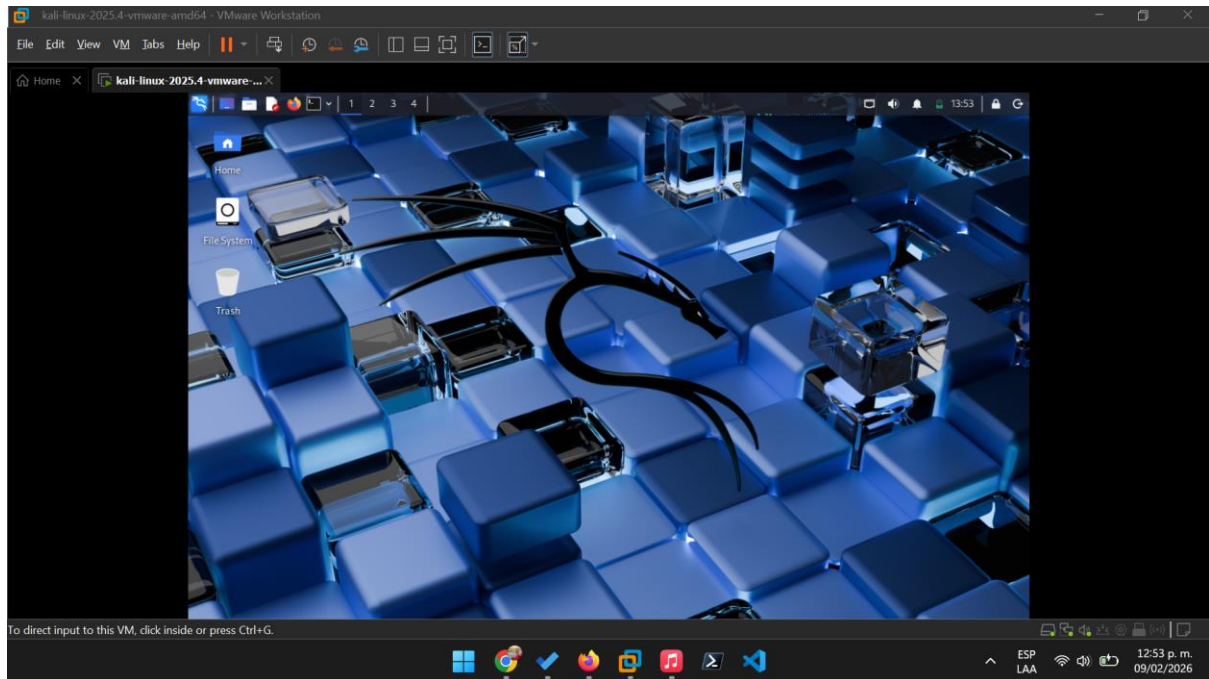
Dirty COW es una vulnerabilidad de escalada de privilegios locales que afectó al kernel de Linux durante casi una década (desde la versión 2.6.22 hasta la 4.8). Su nombre técnico proviene de la función **Copy-On-Write (COW)**, un mecanismo de optimización que utiliza el kernel para gestionar la memoria.

¿Cómo funciona?

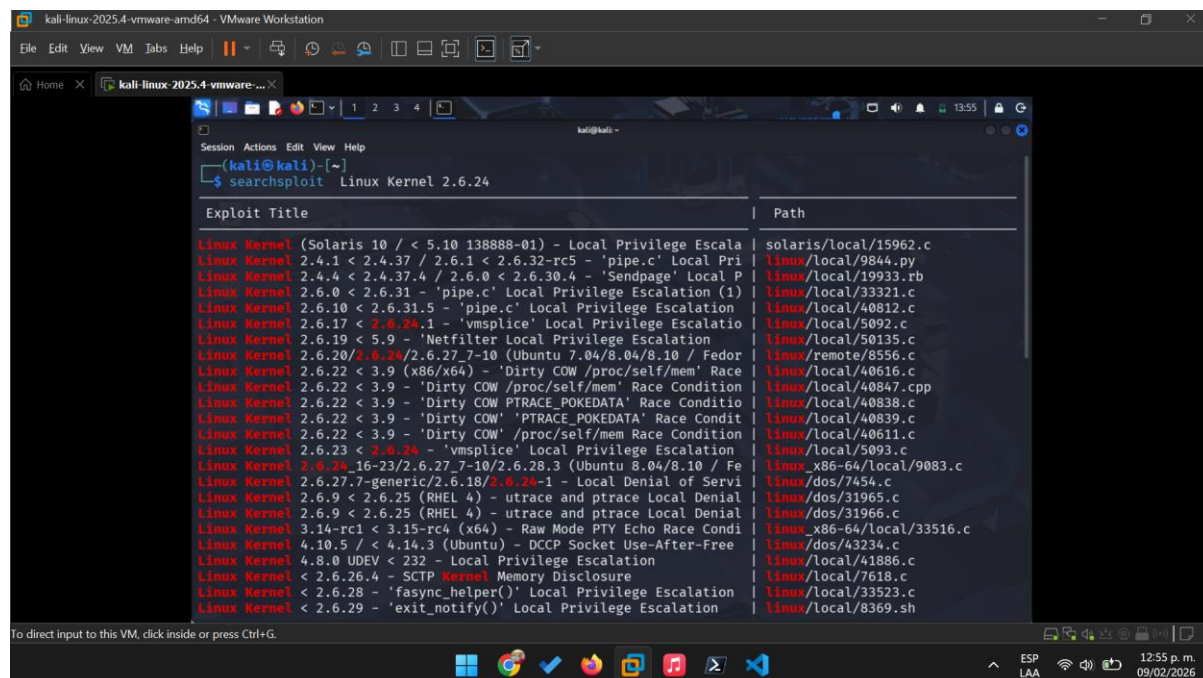
El ataque **aprovecha una condición de carrera** (race condition) en la forma en que el kernel maneja la escritura en memoria mapeada de solo lectura. Normalmente, cuando un proceso intenta modificar un archivo de solo lectura que está compartido, el sistema crea una "**copia**" para ese proceso (el proceso COW). Dirty COW engaña al sistema para que la escritura se realice directamente en el archivo original (el "sucio" o dirty), rompiendo la protección de solo lectura.

DESARROLLO.

Como primer paso encendemos y accedemos a nuestra máquina virtual con kali linux.



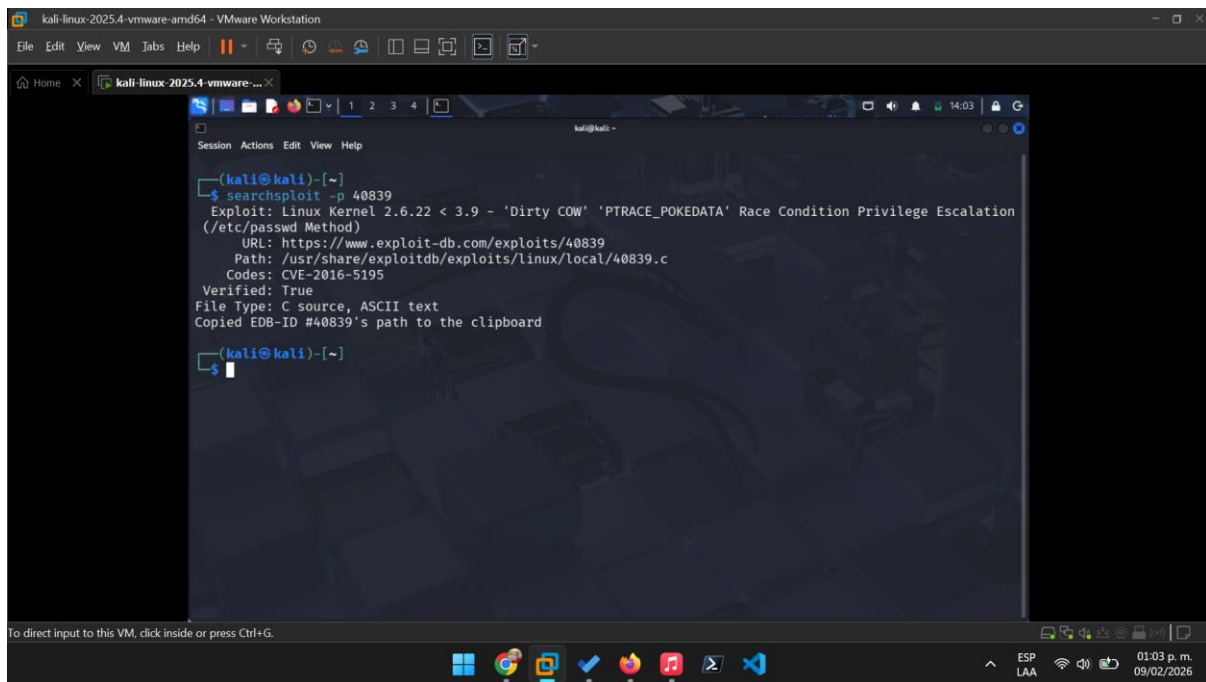
Posteriormente nos dedicamos a abrir la terminal para poder escribir los comandos de searchsploit.



Encontramos gran cantidad de exploits relacionados al kernel linux sin embargo destacaremos los siguientes relacionados con Dirty COW.

- **40816:** Linux Kernel 2.6.22 < 3.9 (x86/x64) - 'Dirty COW' /proc/self/mem Race Condition.
- **40847:** Linux Kernel 2.6.22 < 3.9 - 'Dirty COW /proc/self/mem' Race Condition.
- **40838:** Linux Kernel 2.6.22 < 3.9 - 'Dirty COW PTRACE_POKEDATA' Race Condition.
- **40839:** Linux Kernel 2.6.22 < 3.9 - 'Dirty COW' 'PTRACE_POKEDATA' Race Condition.
- **40611:** Linux Kernel 2.6.22 < 3.9 - 'Dirty COW' /proc/self/mem Race Condition.

Posteriormente nos dedicaremos con el comando **searchsploit -p 40839** a buscar en detalle ese exploit para mostrarnos la ruta completa en el sistema y el enlace directo a la BD web.



The screenshot shows a Kali Linux terminal window within a VMware Workstation. The terminal displays the output of the command `searchsploit -p 40839`. The output provides details about a specific exploit, including its name, target, URL, path, CVE ID, and verification status. The path `/usr/share/exploitdb/exploits/linux/local/40839.c` has been copied to the clipboard.

```
(kali@kali)-[~]
$ searchsploit -p 40839
Exploit: Linux Kernel 2.6.22 < 3.9 - 'Dirty COW' 'Ptrace_PoKedata' Race Condition Privilege Escalation
(/etc/passwd Method)
URL: https://www.exploit-db.com/exploits/40839
Path: /usr/share/exploitdb/exploits/linux/local/40839.c
Codes: CVE-2016-5195
Verified: True
File Type: C source, ASCII text
Copied EDB-ID #40839's path to the clipboard

(kali@kali)-[~]
$
```

RESPUESTA A PREGUNTAS TÉCNICAS.

¿En qué URL se puede encontrar el código fuente de esta vulnerabilidad?

El código fuente se encuentra alojado en la base de datos de Exploit-DB bajo la siguiente URL: <https://www.exploit-db.com/exploits/40839>

¿Cómo puedes ejecutarla y correrla?

Para ejecutar este exploit de manera efectiva, se deben seguir estos pasos técnicos:

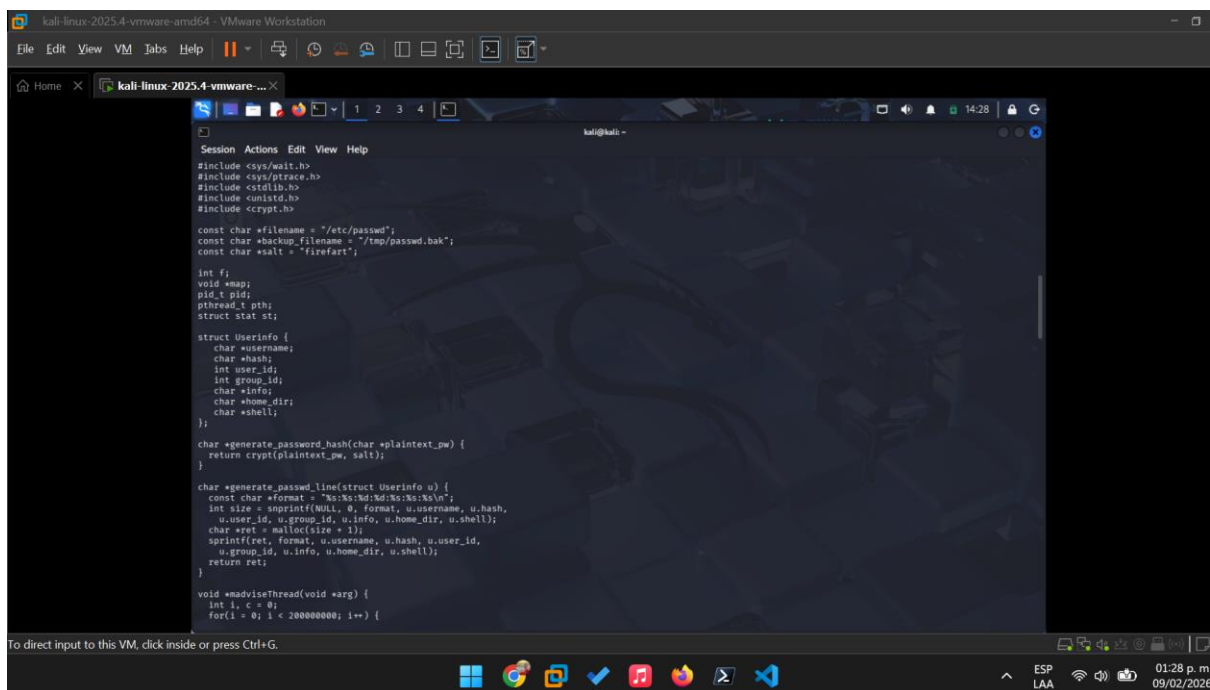
Compilación: Al ser un archivo fuente en lenguaje C (.c), primero se debe compilar usando gcc.

Es fundamental incluir la librería **pthread** para manejar los hilos que causan la condición de carrera: gcc -pthread 40839.c -o dirty -lcrypt

2. Ejecución: Una vez generado el archivo binario ejecutable (dirty), se corre pasando una nueva contraseña como parámetro: /dirty [contraseña_elegida]

Resultado esperado: El exploit aprovecha el fallo Copy-On-Write para sobrescribir archivos protegidos, permitiendo escalar privilegios a root.

Procederemos a hacerle un **cat** al código para analizarlo brevemente.



The image shows a terminal window titled 'kali@kali: ~' with a dark background and a Kali Linux logo watermark. The terminal displays C code for a password hash generator. The code includes headers for `<sys/wait.h>`, `<sys/ptrace.h>`, `<stdlib.h>`, `<unistd.h>`, and `<crypt.h>`. It defines constants for `filename` ("/etc/passwd"), `backup_filename` ("/tmp/passwd.bak"), and `salt` ("firefart"). It declares variables for `int fi`, `void *map`, `pid_t pid`, `pthread_t pth`, `struct stat st`, and a `struct Userinfo` containing `username`, `hash`, `user_id`, `group_id`, `info`, `home_dir`, and `shell`. The code defines `generate_password_hash` to return `crypt(plaintext_pw, salt)`. The `generate_password_line` function formats a line of the password file using `snprintf` and `malloc`. The `main` function is a loop that runs indefinitely, calling `generate_password_line` and `generate_password_hash` for each line.

```
Session Actions Edit View Help

#include <sys/wait.h>
#include <sys/ptrace.h>
#include <stdlib.h>
#include <unistd.h>
#include <crypt.h>

const char *filename = "/etc/passwd";
const char *backup_filename = "/tmp/passwd.bak";
const char *salt = "firefart";

int fi;
void *map;
pid_t pid;
pthread_t pth;
struct stat st;

struct Userinfo {
    char *username;
    char *hash;
    int user_id;
    int group_id;
    char *info;
    char *home_dir;
    char *shell;
};

char *generate_password_hash(char *plaintext_pw) {
    return crypt(plaintext_pw, salt);
}

char *generate_password_line(struct Userinfo u) {
    const char *format = "%s:%s:%d:%d:%s:%s\n";
    int size = snprintf(NULL, 0, format, u.username, u.hash,
        u.user_id, u.group_id, u.info, u.home_dir, u.shell);
    char *ret = malloc(size + 1);
    sprintf(ret, format, u.username, u.hash, u.user_id,
        u.group_id, u.info, u.home_dir, u.shell);
    return ret;
}

void *adviserThread(void *arg) {
    int i, c = 0;
    for(i = 0; i < 200000000; i++) {
```

To direct input to this VM, click inside or press Ctrl+G.

ESP LAA 01:28 p.m. 09/02/2026

Análisis del uso de hilos (pthread) en el exploit

Para que el ataque Dirty COW sea exitoso, es indispensable el uso de la librería pthread.h, ya que la vulnerabilidad se basa en una condición de carrera (race condition).

Creación del hilo (pthread_create): En la función main, el código utiliza pthread_create(&pth, NULL, madviseThread, NULL);. Esto lanza un hilo secundario que ejecuta la función madviseThread de forma paralela al proceso principal.

El hilo saboteador (madvise): Dentro de madviseThread, se ejecuta un bucle que llama constantemente a madvise(map, 100, MADV_DONTNEED);. El objetivo de este hilo es decirle al kernel de Linux: "Ya no necesito esta memoria, puedes liberarla".

La carrera (The Race): Mientras el hilo de `madvise` intenta liberar el mapeo de memoria, el proceso principal intenta escribir en esa misma memoria (que es de solo lectura) usando `ptrace(PTRACE_POKETEXT, ...)`.

El fallo: Debido a un error en el manejo de la función `get_user_pages` del kernel, si el hilo de `madvise` gana la carrera justo en el momento exacto, logra que el kernel escriba los datos en la página física original del archivo `/etc/passwd` en lugar de hacerlo en una copia privada (Copy-on-Write).

MITIGACIÓN Y SOLUCIÓN DE VULNERABILIDAD (SECCIÓN EXTRA).

Dado que Dirty COW aprovecha una debilidad intrínseca en el manejo de memoria del kernel, la solución definitiva no es una configuración de software, sino una actualización del núcleo mismo.

A. Actualización del Kernel (Solución Definitiva)

La forma más segura de corregir esta vulnerabilidad es actualizar el kernel de Linux a una versión que incluya el parche oficial lanzado en octubre de 2016.

En sistemas basados en Debian/Ubuntu/Kali:

sudo apt update && sudo apt upgrade linux-image-amd64

En sistemas basados en RHEL/CentOS:

sudo yum update kernel

B. Uso de SystemTap (Solución Temporal sin Reinicio)

En entornos de misión crítica donde no es posible reiniciar el servidor de inmediato, se puede utilizar un script de SystemTap. Este script actúa como un "parche en vivo" que monitorea las llamadas al sistema y bloquea los intentos de explotación de la función `madvise` que resulten sospechosos.

C. Implementación de Mecanismos de Seguridad Complementarios

Aunque el parche del kernel es vital, se recomienda fortalecer el sistema con:

Políticas de SELinux/AppArmor: Configurar perfiles estrictos que limiten lo que los procesos pueden escribir en memoria, incluso si el kernel tiene fallos.

EDR (Endpoint Detection and Response): Monitorear llamadas inusuales a `ptrace` y `madvise` que ocurran en ráfagas rápidas, lo cual es un indicador típico de este ataque.

CONCLUSIÓN.

El análisis de la vulnerabilidad **Dirty COW (CVE-2016-5195)** nos permite comprender que los sistemas operativos modernos, al ser entidades de software de una complejidad monumental, siempre albergarán vectores de ataque latentes en sus capas más profundas. Esta vulnerabilidad en particular, que permaneció oculta en el kernel de Linux por casi una década, evidencia que incluso los mecanismos de optimización más básicos, como **Copy-on-Write**, pueden ser subvertidos si no se auditan bajo una mentalidad de adversario.

Bajo esta premisa, se vuelve vital reconocer que la seguridad de un ecosistema tan vasto no puede depender de **esfuerzos aislados**. Es imperativo que los equipos de investigación que descubren vulnerabilidades críticas establezcan **canales de cooperación tanto interna como externa**. Esta sinergia debe involucrar activamente a los desarrolladores del núcleo, auditores de seguridad y los mantenedores de las diversas distribuciones de Linux (distros). Solo a través de esta colaboración abierta y transparente es posible que las correcciones se desplieguen en **ciclos pequeños pero constantes**, reduciendo drásticamente la "ventana de exposición" que los atacantes suelen aprovechar.

Finalmente, el caso de Dirty COW reafirma la necesidad de implementar una estrategia de **Defensa en Profundidad**. No podemos confiar la integridad del sistema a un **solo control** (como los permisos de archivos); por el contrario, debemos diseñar entornos donde múltiples capas de seguridad —como el endurecimiento del kernel (Harding), el uso de módulos de seguridad como SELinux o AppArmor, y sistemas de monitoreo de integridad— **trabajen en conjunto**. En ciberseguridad, la fortaleza de un sistema no reside solo en la ausencia de errores, sino en la capacidad colectiva de detectarlos, reportarlos y mitigarlos antes de que se conviertan en una crisis sistémica.

REFERENCIAS BIBLIOGRÁFICAS.

- Hat, R. (2016, 9 noviembre). Understanding and mitigating the dirty cow vulnerability. Red Hat. Recuperado 9 de febrero de 2026, de <https://www.redhat.com/es/blog/understanding-and-mitigating-dirty-cow-vulnerability>.
- De Luz, S. (2023, 24 abril). La conocida vulnerabilidad en Android Dirty Cow no se solucionó del todo. <https://www.redeszone.net/2016/11/29/la-conocida-vulnerabilidad-android-dirty-cow-no-se-soluciono-del-se-espera-lo-haga-diciembre/>